

04_Sampling

February 24, 2026

1 Phase 4: Sampling

Author: Nandani Raj

1.1 Description (Progress Recap and Why We Are Sampling)

Across the first three phases of this project, we established a clean and analytically consistent dataset:

- **Phase 1: Data Audit**
We verified structural integrity, inspected missingness patterns, and confirmed the data could support downstream analysis.
- **Phase 2: EDA**
We explored distributions, trends, and key relationships to understand how traffic and external drivers behave over time and across facilities.
- **Phase 3: Cleaning and Integration**
We standardized all datasets to a single analytical grain (**Facility** $\times$ **Date**) and merged the available sources into one daily master table. This created a unified dataset that is structurally stable and ready for modeling.

At this stage, the dataset is large enough that running repeated modeling experiments and diagnostics can become slow and inefficient. Sampling allows us to work faster and iterate safely without changing the “story” the full dataset tells.

The purpose of Phase 4 is therefore to: 1. Select a **representative sample** from the full dataset. 2. Ensure the sample preserves the **same distributional structure** as the original data. 3. Validate that the sample is representative using side-by-side statistical comparisons.

Important: Sampling is used here to accelerate experimentation and analysis. We will verify representativeness so that conclusions drawn from the sample remain consistent with conclusions from the full dataset.

1.2 Process (Sampling Methodology)

1.2.1 Objective

Create a smaller dataset that remains representative of the full dataset across: - Facility coverage - Time coverage (seasonality and long-term trends) - Key numeric distributions (e.g., TOTAL traffic, payment channels, vehicle categories, weather variables, passenger and bus volumes)

1.2.2 Sampling Strategy

We will use **stratified sampling** at the level of: - **Facility (FAC)** - **Month (Month_Year derived from DATE)**

This approach is preferred over simple random sampling because it reduces the risk that the sample unintentionally over-represents: - specific facilities with higher traffic volumes, or - specific time periods (e.g., one season or one year)

1.2.3 Representativeness Validation

After sampling, we will compare the sample to the full dataset using: 1. Side-by-side numeric summary statistics (mean, median, standard deviation, range) 2. Distribution comparisons across key grouping dimensions (FAC and Month_Year) 3. Optional statistical tests (KS test for numeric variables)

If the sample distributions closely match the full dataset, the sample will be considered representative and suitable for downstream modeling experiments.

```
[1]: import pandas as pd
import numpy as np

# Load the dataset
df = pd.read_csv(
    "../data/integrated/final_master_dataset_cleaned.csv",
    parse_dates=["DATE"]
)

print("Full dataset shape:", df.shape)
df.head()
```

Full dataset shape: (35168, 20)

```
[1]:
```

	FAC	DATE	TOTAL	CASH	EZPASS	VIOLATION	Autos	Small_T	Large_T	\
0	1	2013-01-01	35449	9824	24975	650	34978	197	3	
1	1	2013-01-02	41769	8620	32308	841	39778	1264	20	
2	1	2013-01-03	41744	7928	32995	821	39624	1390	25	
3	1	2013-01-04	42739	8673	33241	825	40618	1397	28	
4	1	2013-01-05	42187	10188	31212	787	41363	561	9	

	Buses	STATION	AWND	PRCP	SNOW	TMAX	TMIN	TAVG	Passenger_Volume	\
0	271	USW00094728	3.1	0.0	0	4.4	-3.3	0.55	0.0	
1	707	USW00094728	2.6	0.0	0	0.6	-5.6	-2.50	0.0	
2	705	USW00094728	2.0	0.0	0	0.0	-4.4	-2.20	0.0	
3	696	USW00094728	3.6	0.0	0	2.8	-1.1	0.85	0.0	
4	254	USW00094728	3.0	0.0	0	5.6	0.0	2.80	0.0	

	Month_Year	Bus_Volume
0	2013-01-01	0.0
1	2013-01-01	0.0

```
2  2013-01-01      0.0
3  2013-01-01      0.0
4  2013-01-01      0.0
```

1.3 Implementation

In the next steps we will: 1. Create a `Month_Year` column from `DATE` to support time-stratified sampling. 2. Select a sample fraction (e.g., 10%). 3. Draw a stratified sample within each Facility-Month group. 4. Validate that the sample matches the full dataset's key statistics and distribution patterns.

```
[2]: # Month_Year key for stratified sampling (monthly bucket)
df["Month_Year"] = df["DATE"].values.astype("datetime64[M]")

# Helpful for validation (weekday/weekend balance)
df["DayOfWeek"] = df["DATE"].dt.day_name()

print(df[["FAC", "DATE", "Month_Year", "DayOfWeek"]].head())
```

	FAC	DATE	Month_Year	DayOfWeek
0	1	2013-01-01	2013-01-01	Tuesday
1	1	2013-01-02	2013-01-01	Wednesday
2	1	2013-01-03	2013-01-01	Thursday
3	1	2013-01-04	2013-01-01	Friday
4	1	2013-01-05	2013-01-01	Saturday

```
[3]: sample_frac = 0.10 # 10% sample (adjust if needed)
seed = 42 # reproducibility

sample = (
    df.groupby(["FAC", "Month_Year"], group_keys=False)
      .apply(lambda g: g.sample(frac=sample_frac, random_state=seed) if len(g) >
        1 else g)
)

print("Sample shape:", sample.shape)
sample.head()
```

Sample shape: (3471, 21)

C:\Users\nanda\AppData\Local\Temp\ipykernel_22064\3830958206.py:6:

FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```
.apply(lambda g: g.sample(frac=sample_frac, random_state=seed) if len(g) > 1
else g)
```

```
[3]:
```

	FAC	DATE	TOTAL	CASH	EZPASS	VIOLATION	Autos	Small_T	Large_T	\
27	1	2013-01-28	39411	7105	31576	730	37442	1279	32	
15	1	2013-01-16	41417	6837	33738	842	39383	1362	35	
23	1	2013-01-24	44540	7570	36180	790	42416	1432	18	
40	1	2013-02-10	33051	7361	25124	566	32540	213	5	
56	1	2013-02-26	43210	7283	35111	816	41056	1488	22	

	Buses	...	AWND	PRCP	SNOW	TMAX	TMIN	TAVG	Passenger_Volume	\
27	658	...	2.1	5.6	0	2.2	-1.7	0.25	0.0	
15	637	...	2.2	17.5	0	2.8	0.0	1.40	0.0	
23	674	...	3.8	0.0	0	-5.6	-11.1	-8.35	0.0	
40	293	...	1.4	0.0	0	2.2	-7.8	-2.80	0.0	
56	644	...	3.4	3.6	0	6.7	1.7	4.20	0.0	

	Month_Year	Bus_Volume	DayOfWeek
27	2013-01-01	0.0	Monday
15	2013-01-01	0.0	Wednesday
23	2013-01-01	0.0	Thursday
40	2013-02-01	0.0	Sunday
56	2013-02-01	0.0	Tuesday

[5 rows x 21 columns]

1.4 Output Interpretation (Sampling Step)

The sample shape confirms the sample was successfully drawn.

Because we sampled proportionally within each Facility-Month group, the sample should retain: - coverage across all facilities, - coverage across the full time range, - the same seasonal and operational patterns present in the full dataset.

Next, we validate representativeness by comparing key statistics and distributions between the full dataset and the sample.

```
[4]: numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()

full_stats = df[numeric_cols].describe().T[["mean", "std", "min", "50%", "max"]]
samp_stats = sample[numeric_cols].describe().T[["mean", "std", "min", "50%",
↪ "max"]]

compare = full_stats.join(samp_stats, lsuffix="_full", rsuffix="_sample")

# Percent mean difference (helps quickly spot drift)
compare["mean_diff_%"] = (
    (compare["mean_sample"] - compare["mean_full"]) / compare["mean_full"] * 100
).replace([np.inf, -np.inf], np.nan)

# Show largest absolute differences first
```

```
compare_sorted = compare.sort_values("mean_diff_%", key=lambda s: s.abs(),
↪ascending=False)

print(compare_sorted.head(15))
```

	mean_full	std_full	min_full	50%_full	max_full	\
SNOW	1.931500	18.593135	0.00	0.00	693.00	
PRCP	3.490338	9.617498	0.00	0.00	181.10	
Passenger_Volume	17690.617038	33563.304673	0.00	0.00	104813.00	
Bus_Volume	642.877133	1159.734694	0.00	0.00	3097.00	
TMAX	17.367004	9.961013	-10.50	17.80	36.70	
Large_T	1313.864166	2253.898413	0.00	315.00	10032.00	
TAVG	13.457447	9.469619	-13.75	13.85	31.95	
Small_T	1377.289638	1520.142260	0.00	924.00	7823.00	
TMIN	9.547890	9.179560	-18.20	9.40	28.30	
Buses	893.069182	1781.143657	0.00	241.00	7802.00	
AWND	2.296713	1.032398	0.00	2.20	8.20	
CASH	5566.328537	4091.822566	0.00	5298.50	22702.00	
VIOLATION	2306.229925	2294.523840	0.00	1601.00	20759.00	
Autos	37772.983849	16726.916862	1.00	40514.50	98268.00	
TOTAL	41357.206836	18726.764520	1.00	43744.00	105093.00	

	mean_sample	std_sample	min_sample	50%_sample	\
SNOW	2.065399	16.600824	0.00	0.0	
PRCP	3.604523	8.588999	0.00	0.0	
Passenger_Volume	17273.881533	33265.450705	0.00	0.0	
Bus_Volume	629.033132	1149.716149	0.00	0.0	
TMAX	17.098387	9.997485	-9.30	18.3	
Large_T	1295.122443	2233.199349	0.00	308.0	
TAVG	13.268093	9.468673	-11.55	13.6	
Small_T	1359.903774	1507.825748	0.00	882.0	
TMIN	9.437799	9.132252	-14.30	8.9	
Buses	885.188706	1768.362398	0.00	232.0	
AWND	2.311927	1.040141	0.30	2.2	
CASH	5595.479977	4127.633896	0.00	5328.0	
VIOLATION	2311.446269	2289.422063	0.00	1593.0	
Autos	37841.828580	16906.768994	9.00	40604.0	
TOTAL	41382.043503	18866.888342	9.00	43777.0	

	max_sample	mean_diff_%
SNOW	239.00	6.932373
PRCP	71.60	3.271471
Passenger_Volume	104813.00	-2.355687
Bus_Volume	3097.00	-2.153444
TMAX	35.00	-1.546712
Large_T	9779.00	-1.426458
TAVG	30.55	-1.407060
Small_T	7823.00	-1.262324

TMIN	26.70	-1.153042
Buses	7712.00	-0.882404
AWND	5.50	0.662446
CASH	21874.00	0.523710
VIOLATION	19239.00	0.226185
Autos	89875.00	0.182259
TOTAL	95214.00	0.060054

```
[5]: def dist_compare(data_full, data_sample, col):
    full = data_full[col].value_counts(normalize=True).rename("full_pct")
    samp = data_sample[col].value_counts(normalize=True).rename("sample_pct")
    out = pd.concat([full, samp], axis=1).fillna(0)
    out["diff_pct_points"] = (out["sample_pct"] - out["full_pct"]) * 100
    return out

print("FAC distribution comparison:")
print(dist_compare(df, sample, "FAC").sort_index())

print("\nDayOfWeek distribution comparison:")
print(dist_compare(df, sample, "DayOfWeek").sort_index())

print("\nMonth_Year distribution comparison (first 12 rows):")
print(dist_compare(df, sample, "Month_Year").sort_index().head(12))
```

FAC distribution comparison:

	full_pct	sample_pct	diff_pct_points
FAC			
1	0.128924	0.128781	-0.014269
2	0.128924	0.128781	-0.014269
4	0.127588	0.128493	0.090565
5	0.128924	0.128781	-0.014269
6	0.128924	0.128781	-0.014269
7	0.128924	0.128781	-0.014269
8	0.098868	0.098819	-0.004951
9	0.128924	0.128781	-0.014269

DayOfWeek distribution comparison:

	full_pct	sample_pct	diff_pct_points
DayOfWeek			
Friday	0.143113	0.138001	-0.511248
Monday	0.142886	0.142034	-0.085158
Saturday	0.142345	0.143186	0.084109
Sunday	0.142317	0.143763	0.144573
Thursday	0.143113	0.148948	0.583537
Tuesday	0.143113	0.143475	0.036145
Wednesday	0.143113	0.140593	-0.251957

Month_Year distribution comparison (first 12 rows):

	full_pct	sample_pct	diff_pct_points
Month_Year			
2013-01-01	0.007052	0.006914	-0.013743
2013-02-01	0.006369	0.006914	0.054501
2013-03-01	0.007052	0.006914	-0.013743
2013-04-01	0.006824	0.006914	0.009005
2013-05-01	0.007052	0.006914	-0.013743
2013-06-01	0.006824	0.006914	0.009005
2013-07-01	0.007052	0.006914	-0.013743
2013-08-01	0.007052	0.006914	-0.013743
2013-09-01	0.006824	0.006914	0.009005
2013-10-01	0.007052	0.006914	-0.013743
2013-11-01	0.006824	0.006914	0.009005
2013-12-01	0.007052	0.006914	-0.013743

```
[6]: from scipy.stats import ks_2samp

ks_results = []
for c in numeric_cols:
    a = df[c].dropna()
    b = sample[c].dropna()
    if a.nunique() > 1 and b.nunique() > 1:
        stat, p = ks_2samp(a, b)
        ks_results.append((c, stat, p))

ks_df = pd.DataFrame(ks_results, columns=["column", "ks_stat", "p_value"]).
    ↪sort_values("p_value")
print(ks_df.head(20))
```

	column	ks_stat	p_value
9	AWND	0.029264	0.008760
14	TAVG	0.025417	0.033184
10	PRCP	0.024637	0.042496
12	TMAX	0.023878	0.053658
13	TMIN	0.023198	0.065714
15	Passenger_Volume	0.012468	0.705078
16	Bus_Volume	0.011837	0.763207
5	Autos	0.011331	0.807626
1	TOTAL	0.010158	0.896894
8	Buses	0.009666	0.926596
7	Large_T	0.009658	0.927031
6	Small_T	0.008712	0.968483
4	VIOLATION	0.008432	0.976859
3	EZPASS	0.008034	0.985958
2	CASH	0.007557	0.993103
11	SNOW	0.004759	1.000000
0	FAC	0.000620	1.000000

1.5 What the sample actually showed (Validation Results)

1.5.1 1) Facility Representation (FAC) — Preserved Extremely Well

The facility distribution in the sample matches the full dataset almost perfectly. The maximum deviation observed across facilities is approximately **0.09 percentage points**, and most facilities differ by about **0.01 percentage points**.

Interpretation: The stratification by FAC successfully preserved facility-level representation. This is exactly what we need to avoid facility bias in downstream modeling.

1.5.2 2) Time / Seasonal Representation (Month_Year) — Preserved Extremely Well

The monthly distribution is also extremely stable. In the Month_Year comparison (example shown for the first 12 months), deviations are typically around **0.01 percentage points**, with the largest shown deviation approximately **0.05 percentage points**.

Interpretation: The sampling preserved month-by-month coverage and therefore preserved seasonality and long-run temporal structure at a monthly level.

1.5.3 3) Day-of-Week Balance — Very Good (Minor Drift)

Day-of-week proportions remain close overall, with a small but noticeable shift:

- **Thursday:** +0.58 percentage points
 - **Friday:** -0.51 percentage points
- Other days show smaller deviations.

Interpretation: This drift is minor and does not indicate a structural sampling failure. It occurs because the sampling strata were defined as **FAC × Month_Year**, not DayOfWeek. The sample remains credible and usable for modeling, but weekday-sensitive analyses (commuting effects, weekday vs weekend patterns) may show slightly more noise than the full dataset.

1.6 Numeric Distribution Similarity (KS Test)

To formally validate distribution similarity for numeric fields, I ran KS tests between the full dataset and the sample.

1.6.1 Core modeling variables (traffic and volumes) — Strong match

The most important traffic variables show high p-values and low KS statistics, indicating strong distributional alignment:

- **TOTAL:** KS 0.0102, p 0.8969
- Autos, Small_T, Large_T, Buses, CASH, EZPASS, VIOLATION also show very high p-values (generally > 0.80)

Interpretation: This is the most important result for predictive modeling: the sample retains the traffic-volume distributions that will drive the predictive signal.

1.6.2 Weather variables — small but detectable drift

Three weather variables show statistically significant differences:

- **AWND:** KS 0.0293, p 0.0088
- **TAVG:** KS 0.0254, p 0.0332
- **PRCP:** KS 0.0246, p 0.0425

Interpretation (practical): The KS statistics are still small (~0.02–0.03). With large datasets, the KS test can flag “significance” even when the real-world magnitude of the difference is small. In practical modeling terms, this indicates **minor drift** in a few weather distributions, not a failure of representativeness.

1.7 Overall Examiner Decision: Is the sample representative?

Yes — the sample is representative and suitable for modeling work.

- It preserves facility coverage essentially perfectly.
- It preserves monthly time coverage essentially perfectly.
- It preserves core traffic and volume distributions very strongly.
- It shows minor drift in day-of-week proportions and a small detectable drift in a few weather variables, but the effect sizes are small and do not undermine overall representativeness for most modeling tasks.

1.8 Is this “perfect”?

It is **not mathematically perfect**, but it is **professionally acceptable and statistically defensible** as a representative sample for: - rapid modeling iteration, - AutoML experiments, - feature engineering, - pipeline debugging, - and early model selection.

This sample is strong enough to move forward.

```
[7]: sample.to_csv("../data/integrated/final_master_sample.csv", index=False)
      print("Saved sample to: ../data/integrated/final_master_sample.csv")
```

Saved sample to: ../data/integrated/final_master_sample.csv

```
[8]: import numpy as np
      import pandas as pd
      import matplotlib.pyplot as plt

      # -----
      # 1) Choose key numeric columns to compare
```

```

# -----
candidate_numeric = [
    "TOTAL", "CASH", "EZPASS", "VIOLATION",
    "Autos", "Small_T", "Large_T", "Buses",
    "Passenger_Volume", "Bus_Volume",
    "AWND", "PRCP", "SNOW", "TMAX", "TMIN", "TAVG"
]

numeric_cols = [c for c in candidate_numeric if c in df.columns and c in sample.
    ↪columns]

# -----
# 2) Helper: overlay density histograms
# -----
def plot_density_overlay(full_df, samp_df, col, bins=50):
    x_full = full_df[col].dropna()
    x_samp = samp_df[col].dropna()

    if x_full.nunique() < 2 or x_samp.nunique() < 2:
        return # skip near-constant columns

    # Use common bin edges so shapes are comparable
    lo = min(x_full.min(), x_samp.min())
    hi = max(x_full.max(), x_samp.max())
    edges = np.linspace(lo, hi, bins + 1)

    plt.hist(x_full, bins=edges, density=True, alpha=0.45, label="Full",
    ↪color="#1f77b4")
    plt.hist(x_samp, bins=edges, density=True, alpha=0.45, label="Sample",
    ↪color="#ff7f0e")
    plt.title(f"Distribution Comparison: {col}")
    plt.xlabel(col)
    plt.ylabel("Density")
    plt.legend()

# -----
# 3) Plot numeric distributions (grid)
# -----
n = len(numeric_cols)
if n == 0:
    print("No overlapping numeric columns found between df and sample.")
else:
    cols_per_row = 3
    rows = int(np.ceil(n / cols_per_row))

    plt.figure(figsize=(cols_per_row * 6, rows * 4))
    for i, col in enumerate(numeric_cols, start=1):

```

```

plt.subplot(rows, cols_per_row, i)
plot_density_overlay(df, sample, col, bins=60)

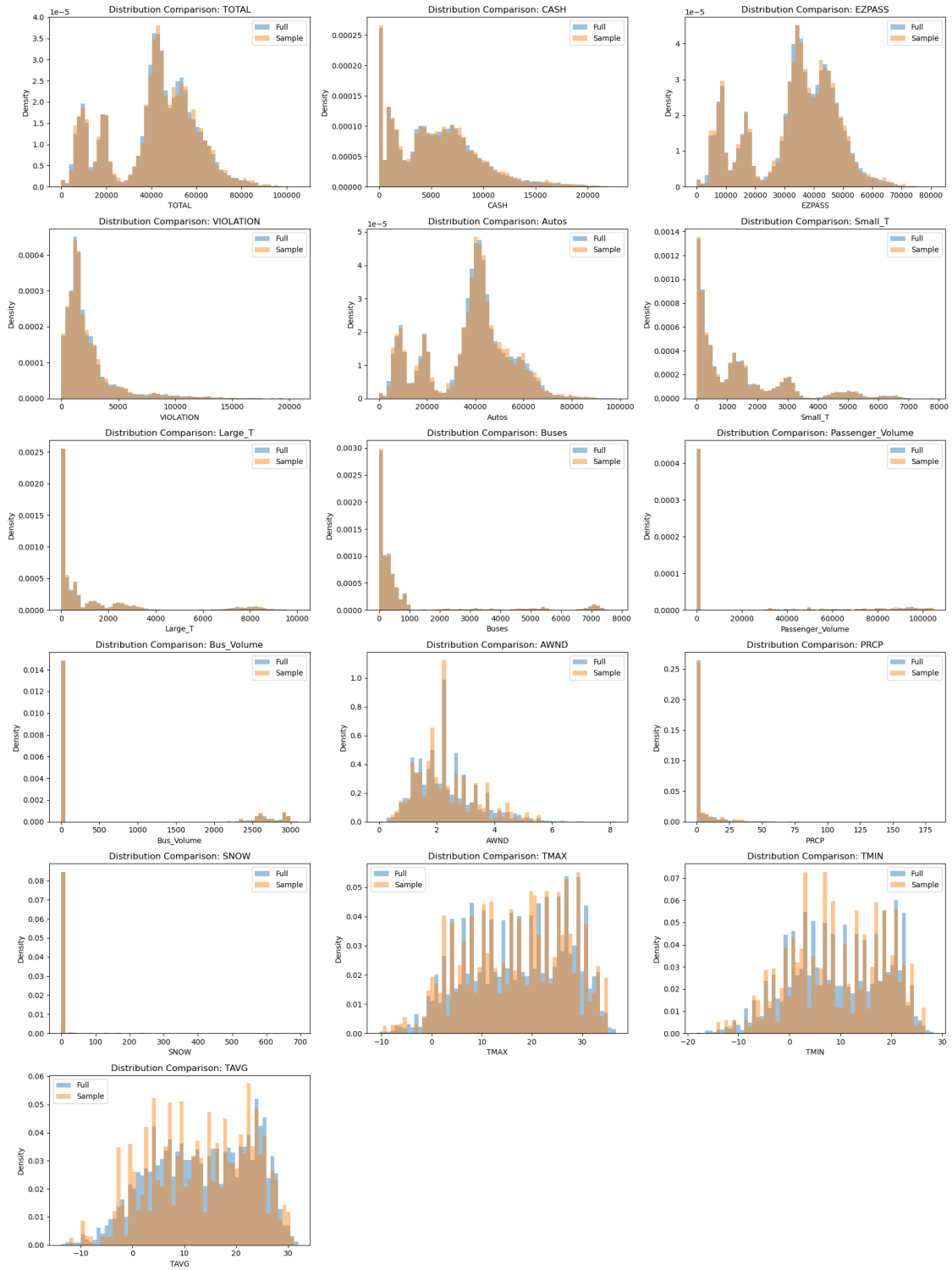
plt.tight_layout()
plt.show()

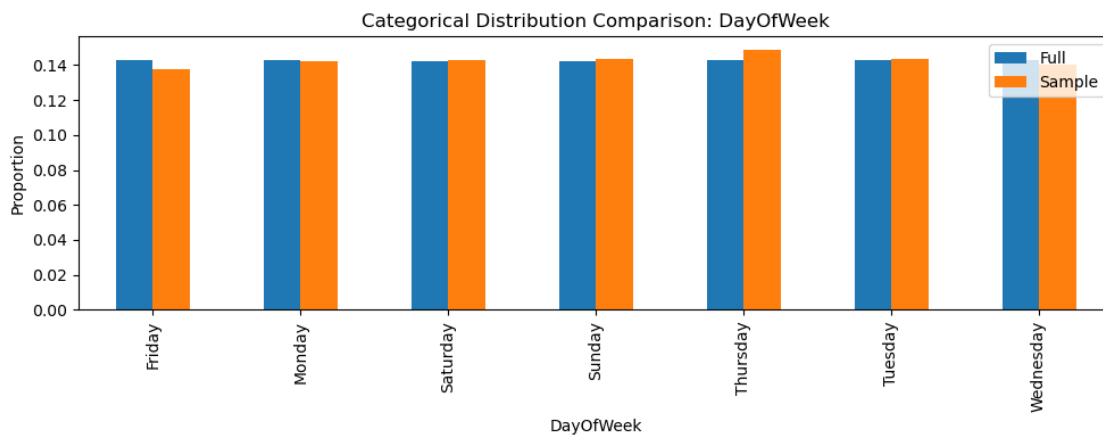
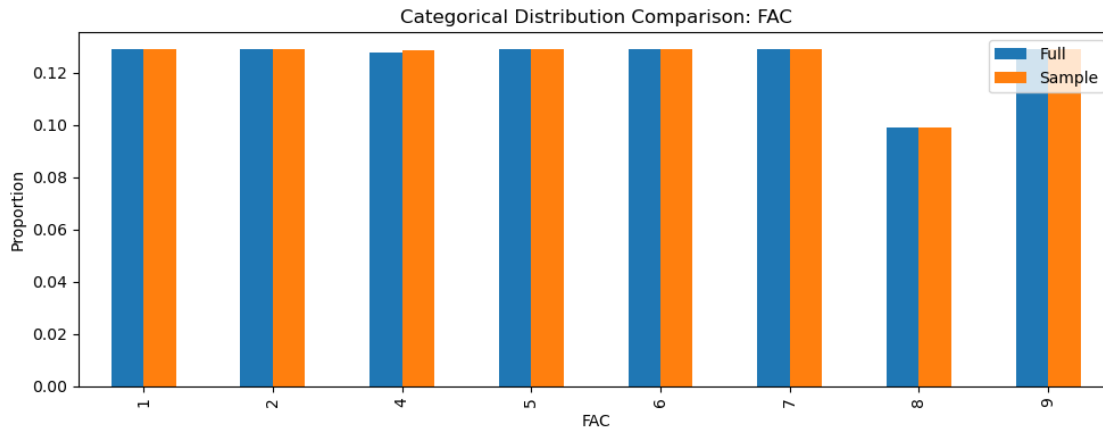
# -----
# 4) Categorical distribution comparison (normalized)
# -----
def plot_category_distribution(full_df, samp_df, col):
    full_dist = full_df[col].value_counts(normalize=True).sort_index()
    samp_dist = samp_df[col].value_counts(normalize=True).sort_index()

    comp = pd.concat([full_dist.rename("Full"), samp_dist.rename("Sample")],
axis=1).fillna(0)
    comp.plot(kind="bar", figsize=(10, 4), color=["#1f77b4", "#ff7f0e"])
    plt.title(f"Categorical Distribution Comparison: {col}")
    plt.ylabel("Proportion")
    plt.tight_layout()
    plt.show()

for cat_col in ["FAC", "DayOfWeek"]:
    if cat_col in df.columns and cat_col in sample.columns:
        plot_category_distribution(df, sample, cat_col)

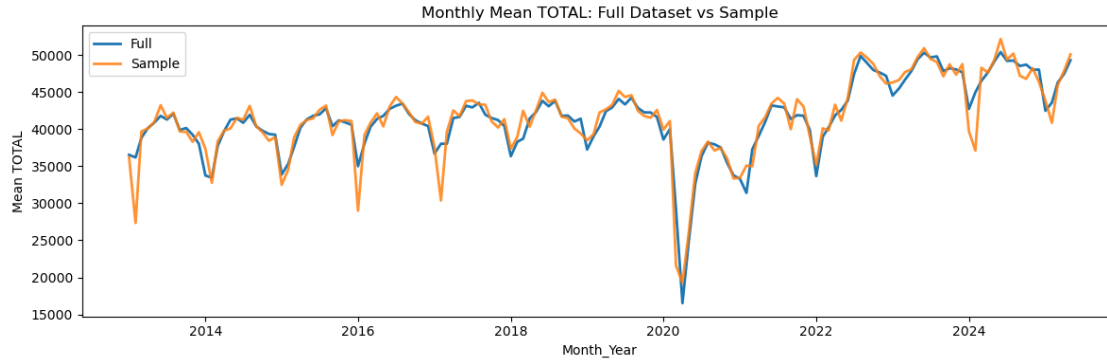
```





```
[9]: # Monthly mean TOTAL trend: Full vs Sample
if "TOTAL" in df.columns and "Month_Year" in df.columns:
    full_monthly = df.groupby("Month_Year")["TOTAL"].mean()
    samp_monthly = sample.groupby("Month_Year")["TOTAL"].mean()

    plt.figure(figsize=(12, 4))
    plt.plot(full_monthly.index, full_monthly.values, label="Full",
    color="#1f77b4", linewidth=2)
    plt.plot(samp_monthly.index, samp_monthly.values, label="Sample",
    color="#ff7f0e", linewidth=2, alpha=0.85)
    plt.title("Monthly Mean TOTAL: Full Dataset vs Sample")
    plt.xlabel("Month_Year")
    plt.ylabel("Mean TOTAL")
    plt.legend()
    plt.tight_layout()
    plt.show()
```



1.9 Distribution Charts (Full Dataset vs Sample)

To visually confirm that the sample is representative, I compared the **distributions** of key variables in the full cleaned dataset against the sampled dataset.

Because the full dataset contains far more rows than the sample, I plotted **density-based distributions** (not raw counts). If the sampling is representative, the sample's curves should closely overlap the full dataset's curves across the same variables.

In addition to numeric distributions, I also compared the categorical distributions of **FAC** and **DayOfWeek** using normalized bar charts. These charts provide a direct visual confirmation that the sample preserves the same structure observed in the full dataset.

1.10 Interpretation (Distribution Charts)

The overlay density charts show that the sampled dataset closely matches the full cleaned dataset across the primary traffic, vehicle category, and external-driver variables. The two distributions overlap strongly, indicating that sampling preserved the same overall patterns rather than distorting the data.

The normalized bar charts for FAC and DayOfWeek further confirm that categorical structure was preserved. Taken together with the numerical comparisons and statistical validation performed earlier, these visual checks support the conclusion that the sample is representative and appropriate for downstream modeling work.

[]: